Thinking in Objects

CSE 114: Introduction to Object-Oriented Programming

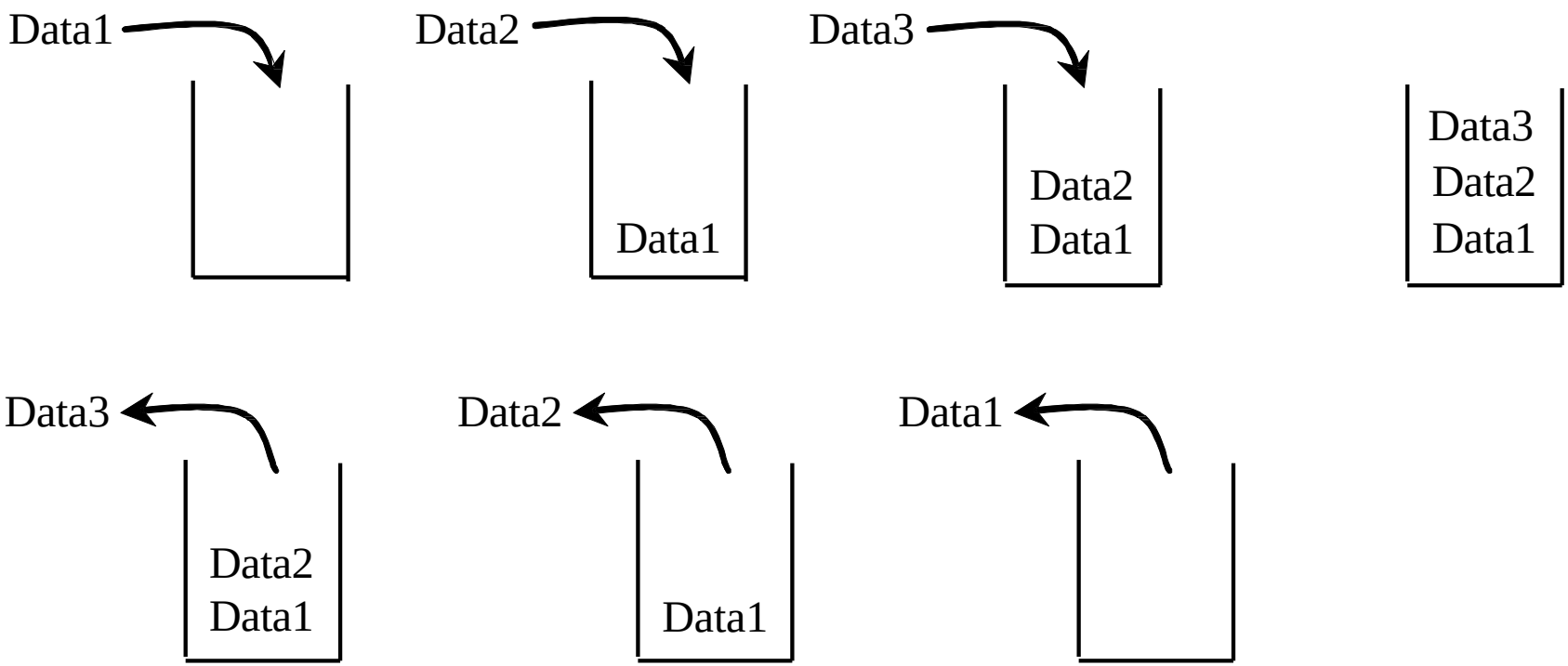
Contents

- Immutable Objects and Classes
- Scope of Variables and Default values
- The **`this`** Keyword
- Calling Overloaded Constructors
- Class Abstraction and Encapsulation
  - Designing and implementing the Loan Class
  - Designing and implementing the BMI Class
  - Designing and implementing the Course Class
  - Designing and implementing the StackOfIntegers Class
- The String Class in detail
  - Regular Expressions
  - Command-Line Parameters
- StringBuilder and StringBuffer
- The Character Class
- Designing Classes

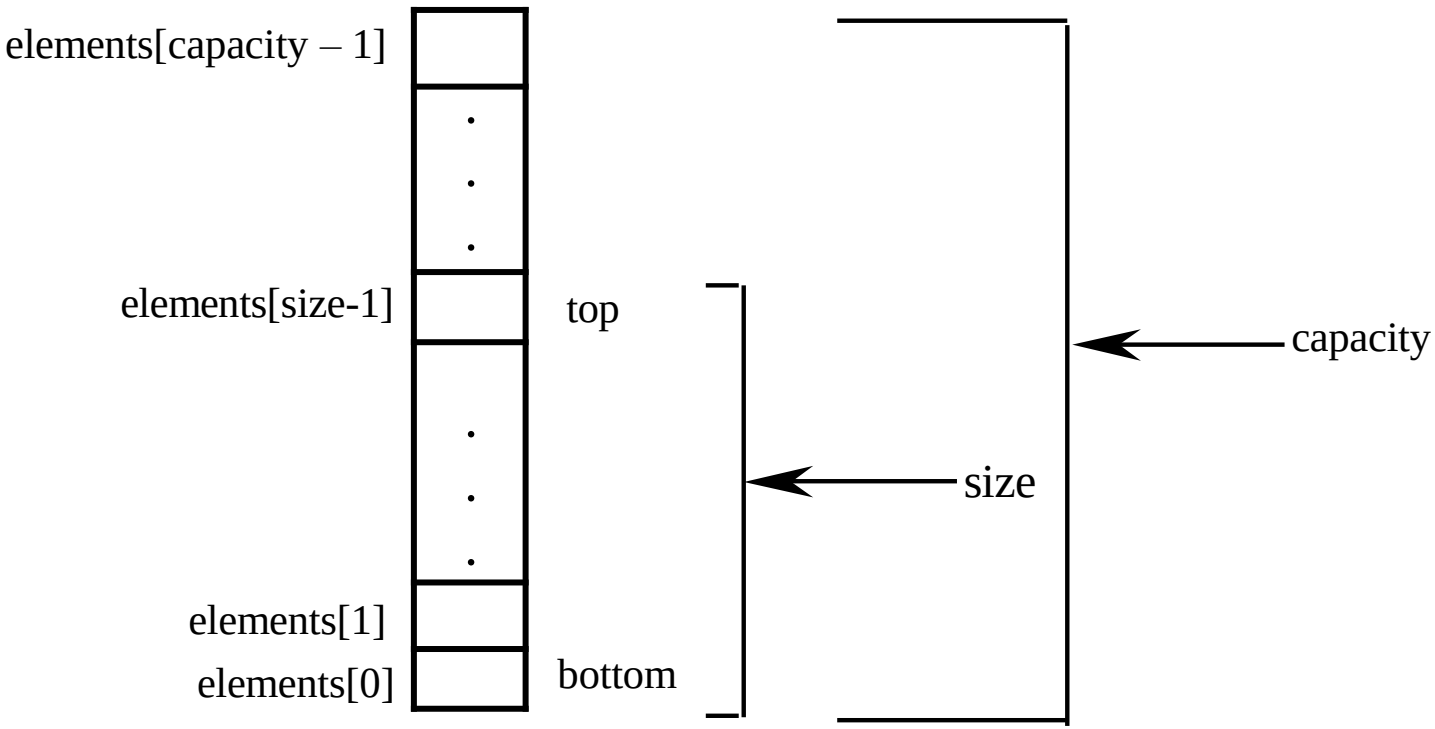
Example: The `StackOfIntegers` Class

StackOfIntegers	
-elements: int[]	An array to store integers in the stack.
-size: int	The number of integers in the stack.
+StackOfIntegers()	Constructs an empty stack with a default capacity of 16.
+StackOfIntegers(capacity: int)	Constructs an empty stack with a specified capacity.
+empty(): boolean	Returns true if the stack is empty.
+peek(): int	Returns the integer at the top of the stack without removing it from the stack.
+push(value: int): int	Stores an integer into the top of the stack.
+pop(): int	Removes the integer at the top of the stack and returns it.
+getSize(): int	Returns the number of elements in the stack.

Designing the `StackOfIntegers` Class



Implementing `StackOfIntegers` Class



```java
public class StackOfIntegers {
  private int[] elements;
  private int size;
  public static final int DEFAULT_CAPACITY = 16;
  public StackOfIntegers() {
    this(DEFAULT_CAPACITY);
  }
  public StackOfIntegers(int capacity) {
    elements = new int[capacity];
  }
  public void push(int value) {
    if (size >= elements.length) {
      int[] temp = new int[elements.length * 2];
      System.arraycopy(elements, 0, temp, 0, elements.length);
      elements = temp;
    }
    elements[size++] = value;
  }
  public int pop() {
    return elements[--size];
  }
  public int peek() {
    return elements[size - 1];
  }
  public int getSize() {
    return size;
  }}
```

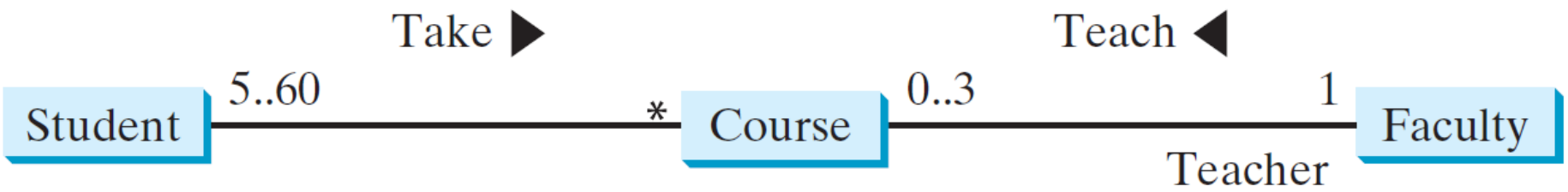
Class Relationships

- Association
- Aggregation
- Composition
- Inheritance (Chapter 13)

Association: is a general binary relationship that describes an activity between two classes.

- For example, a student taking a course is an association between the **Student** class and the **Course** class, and a faculty member teaching a course is an association between the **Faculty** class and the **Course** class. These associations can be represented in UML graphical notation, as shown below.

Class Relationships



An association is illustrated by a solid line between two classes with an optional label that describes the relationship. The labels are *Take* and *Teach*. Each relationship may have an optional small black triangle that indicates the direction of the relationship. In this figure, the direction indicates that a student takes a course (as opposed to a course taking a student).

Class Relationships

- Each class involved in an association may specify a *multiplicity*, which is placed at the side of the class to specify how many of the class's objects are involved in the relationship in UML.
- A multiplicity could be a number or an interval that specifies how many of the class's objects are involved in the relationship. The character ***** means an unlimited number of objects, and the interval **m..n** indicates that the number of objects is between **m** and **n**, inclusively.

Class Relationships

- Each student may take any number of courses, and each course must have at least 5 and at most 60 students. Each course is taught by only 1 faculty member, and a faculty member may teach from 0 to 3 courses per semester.

Implementation

In Java code, you can implement associations by using data fields and methods.

```java
public class Student {
  private Course[]
    courseList;

  public void addCourse(
    Course s) { ... }
}
```

```java
public class Course {
  private Student[]
    classList;
  private Faculty faculty;

  public void addStudent(
    Student s) { ... }

  public void setFaculty(
    Faculty faculty) { ... }
}
```

```java
public class Faculty {
  private Course[]
    courseList;

  public void addCourse(
    Course c) { ... }
}
```

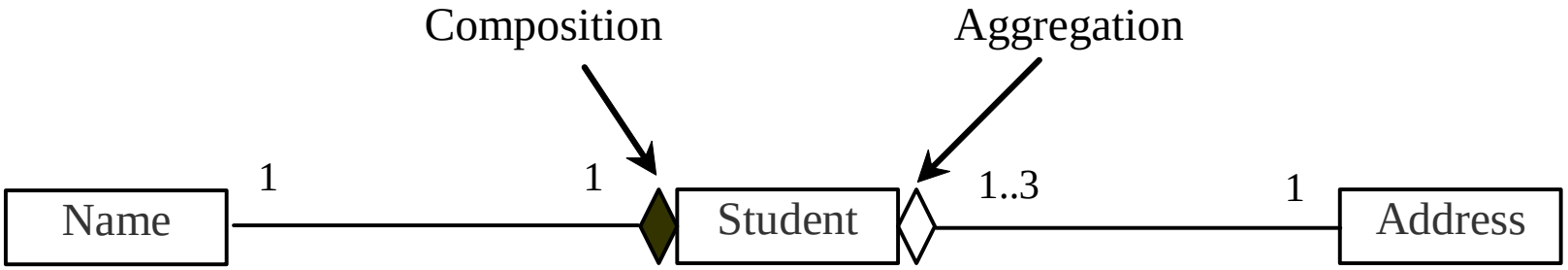
Aggregation and Composition

Aggregation is a special form of association that represents an ownership relationship between two objects. Aggregation models *has-a* relationships. The owner object is called an *aggregating object*, and its class is called an *aggregating class*. The subject object is called an *aggregated object*, and its class is called an *aggregated class*.

An object can be owned by several other aggregating objects. If an object is exclusively owned by an aggregating object, the relationship between the object and its aggregating object is referred to as a *composition*. Composition is actually a special case of the aggregation relationship.

Aggregation and Composition

"a student has a name" is a composition relationship between the **Student** class and the **Name** class, whereas "a student has an address" is an aggregation relationship between the **Student** class and the **Address** class, since an address can be shared by several students. In UML, a filled diamond is attached to an aggregating class (in this case, **Student**) to denote the composition relationship with an aggregated class (**Name**), and an empty diamond is attached to an aggregating class (**Student**) to denote the aggregation relationship with an aggregated class (**Address**),



Class Representation

An aggregation relationship is usually represented as a data field in the aggregating class. For example, the relationship shown above can be represented as follows:

```
public class Name {
  ...
}
```

Aggregated class

```
public class Student {
  private Name name;
  private Address address;

  ...
}
```

Aggregating class

```
public class Address {
  ...
}
```

Aggregated class

Aggregation or Composition

Since aggregation and composition relationships are represented using classes in similar ways, many texts don’t differentiate them and call both compositions.

Wrapper Classes

- The primitive data type values are not objects in Java. However, many Java methods require the use of objects as arguments.

- Java offers a convenient way to incorporate, or wrap, a primitive data type into an object (e.g., wrapping **int** into the **Integer** class, wrapping **double** into the **Double** class, and wrapping **char** into the **Character** class,).

- By using a wrapper class, you can process primitive data type values as objects.

Wrapper Classes

- Java provides **Boolean**, **Character**, **Double**, **Float**, **Byte**, **Short**, **Integer**, and **Long** wrapper classes in the **java.lang** package for primitive data types. The **Boolean** class wraps a Boolean value **true** or **false**.
- Numeric wrapper classes are very similar to each other. Each contains the methods **doubleValue()**, **floatValue()**, **intValue()**, **longValue()**, **shortValue()**, and **byteValue()**. These methods "convert" objects into primitive type values.
- The wrapper classes do not have no-arg constructors. The instances of all wrapper classes are immutable; this means that, once the objects are created, their internal values cannot be changed.

The Integer and Double Classes

java.lang.Integer

```
-value: int
+MAX_VALUE: int
+MIN_VALUE: int

+Integer(value: int)
+Integer(s: String)
+byteValue(): byte
+shortValue(): short
+intValue(): int
+longVlaue(): long
+floatValue(): float
+doubleValue():double
+compareTo(o: Integer): int
+toString(): String
+valueOf(s: String): Integer
+valueOf(s: String, radix: int): Integer
+parseInt(s: String): int
+parseInt(s: String, radix: int): int
```

java.lang.Double

```
-value: double
+MAX_VALUE: double
+MIN_VALUE: double

+Double(value: double)
+Double(s: String)
+byteValue(): byte
+shortValue(): short
+intValue(): int
+longVlaue(): long
+floatValue(): float
+doubleValue():double
+compareTo(o: Double): int
+toString(): String
+valueOf(s: String): Double
+valueOf(s: String, radix: int): Double
+parseDouble(s: String): double
+parseDouble(s: String, radix: int): double
```

The `Integer` and `Double` Classes

The following statements display the maximum integer (2,147,483,647), the minimum positive float (1.4E–45), and the maximum double floating-point number (1.79769313486231570e + 308d).

```
System.out.println("The maximum integer is " + Integer.MAX_VALUE);

System.out.println("The minimum positive float is " +
Float.MIN_VALUE);

System.out.println( "The maximum double-precision floating-
point number is " + Double.MAX_VALUE);
```

The `Integer` and `Double` Classes

Each numeric wrapper class contains the methods **doubleValue()**, **floatValue()**, **intValue()**, **longValue()**, and **shortValue()** for returning a **double**, **float**, **int**, **long**, or **short** value for the wrapper object. For example,

new Double(12.4).intValue() returns **12**;
new Integer(12).doubleValue() returns **12.0**;

Numeric Wrapper Class Constructors

You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value. The constructors for Integer and Double are:

```
public Integer(int value)

public Integer(String s)

public Double(double value)

public Double(String s)
```

Conversion Methods

Each numeric wrapper class implements the abstract methods doubleValue, floatValue, intValue, longValue, and shortValue, which are defined in the Number class. These methods “convert” objects into primitive type values.

The Static valueOf Methods

The numeric wrapper classes have a useful class method, valueOf(String s). This method creates a new object initialized to the value represented by the specified string. For example:

```
Double doubleObject = Double.valueOf("12.4");
Integer integerObject = Integer.valueOf("12");
```

The Methods for Parsing Strings into Numbers

You have seen the parseInt method in the Integer class to parse a numeric string into an int value and the parseDouble method in the Double class to parse a numeric string into a double value (in Chapter 4).

```
int intValue = Integer.parseInt(intString);
```

Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value. See examples in the book.

Automatic Conversion Between Primitive Types and Wrapper Class Types

A primitive type value can be automatically converted to an object using a wrapper class, and vice versa, depending on the context.

Converting a primitive value to a wrapper object is called *boxing*. The reverse conversion is called *unboxing*. Java allows primitive types and wrapper classes to be converted automatically.

The compiler will automatically box a primitive value that appears in a context requiring an object, and will unbox an object that appears in a context requiring a primitive value. This is called *autoboxing* and *autounboxing*.

Automatic Conversion Between Primitive Types and Wrapper Class Types

```
Integer[] intArray = {new Integer(2),
  new Integer(4), new Integer(3)};
```

(a)

Equivalent

```
Integer[] intArray = {2, 4, 3};
```

New JDK 1.5 boxing

(b)

```
Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);
```

Unboxing

BigInteger and BigDecimal

If you need to compute with very large integers or high precision floating-point values, you can use the BigInteger and BigDecimal classes in the java.math package.
An instance of **BigInteger** can represent an integer of any size. You can use **new BigInteger(String)** and **new BigDecimal(String)** to create an instance of **BigInteger** and **BigDecimal**, use the **add**, **subtract**, **multiply**, **divide**, and **remainder** methods to perform arithmetic operations, and use the **compareTo** method to compare two big numbers.

The String class in detail

- Remember the String class discussed at the beginning of the semester?
  - Constructing Strings:

```
String newString = new String(stringLiteral);
```

    - Example:

```
String message = new String("Welcome to Java");
```

  - Since strings are used frequently, Java provides a shorthand initializer for creating a string:

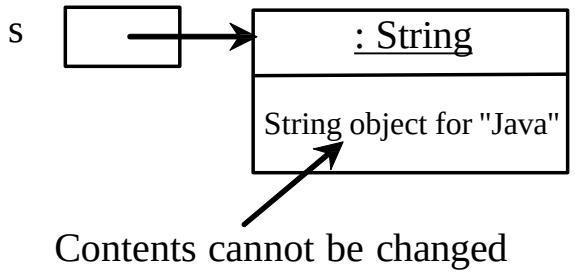
```
String message = "Welcome to Java";
                    // an Interned String
```

Strings Are Immutable

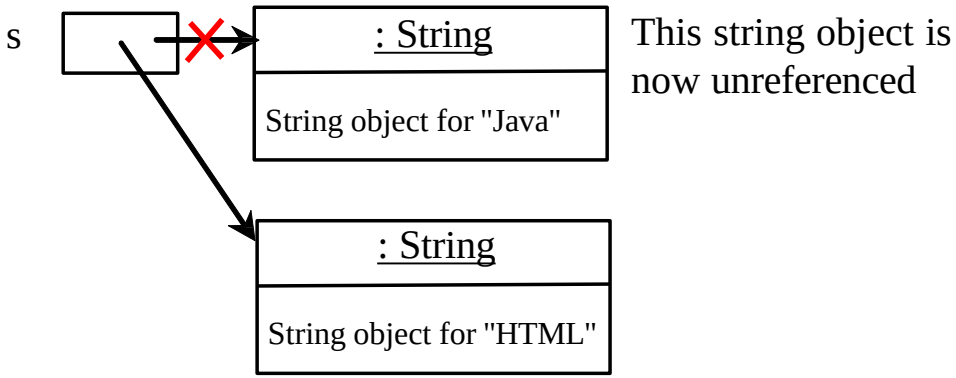
- A String object is immutable; its contents cannot be changed

```
String s = "Java";
s = "HTML";
```

After executing `String s = "Java";`



After executing `s = "HTML";`



Interned Strings

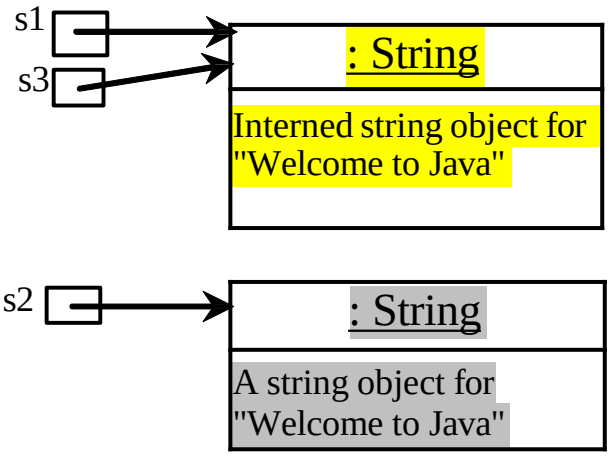
- ***String interning*** is a method of storing only one copy of each distinct compile-time constant/explicit string in the source code stored in a string intern pool (e.g., s1 and s3).
- If we use the **new** operator, then a new object is created in heap (e.g., s2).

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



displays: `s1 == s3` is true

`s1 == s2` is false

The `String` Class methods

- Compare strings (equals, compareTo)
- Obtaining String length
- Retrieving Individual Characters in a string
- String Concatenation (concat)
- Substrings (substring(index), substring(start, end))
- Finding a Character or a Substring in a String
- Matching, Replacing and Splitting by Patterns
- Converting Characters and Numeric Values to Strings
- Command-Line Parameters

String Comparisons

- `equals(Object object)`:

```
String s1 = new String("Welcome");
String s2 = "Welcome";

if (s1.equals(s2)){ // true
  // s1 and s2 have the same contents
}

if (s1 == s2) { // false
  // s1 and s2 have different references
}
```

String Comparisons

- `equals(Object object)`:

```
String s1 = "Welcome";
String s2 = "Welcome";

if (s1.equals(s2)){ // true
  // s1 and s2 have the same contents
}

if (s1 == s2) { // true
  // s1 and s2 have the same reference
}
```

String Comparisons

- `compareTo(Object object):`

```
String s1 = new String("Welcome");
String s2 = new String("Welcome");

 if (s1.compareTo(s2) > 0) {
   // s1 is greater than s2
 }else if (s1.compareTo(s2) == 0) {
   // s1 and s2 have the same contents
 }else{
   // s1 is less than s2
}
```

String Comparisons

java.lang.String	
+equals(s1: String): boolean	Returns true if this string is equal to string s1.
+equalsIgnoreCase(s1: String): boolean	Returns true if this string is equal to string s1 case-insensitive.
+compareTo(s1: String): int	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
+compareToIgnoreCase(s1: String): int	Same as compareTo except that the comparison is case-insensitive.
+regionMatches(toffset: int, s1: String, offset: int, len: int): boolean	Returns true if the specified subregion of this string exactly matches the specified subregion in string s1.
+regionMatches(ignoreCase: boolean, toffset: int, s1: String, offset: int, len: int): boolean	Same as the preceding method except that you can specify whether the match is case-sensitive.
+startsWith(prefix: String): boolean	Returns true if this string starts with the specified prefix.
+endsWith(suffix: String): boolean	Returns true if this string ends with the specified suffix.

String Length, Characters, and Combining Strings

java.lang.String	
+length(): int	Returns the number of characters in this string.
+charAt(index: int): char	Returns the character at the specified index from this string.
+concat(s1: String): String	Returns a new string that concatenate this string with string s1.

Retrieving Individual Characters in a String

- Use **`message.charAt(index)`** → **`char`**
- Index starts from 0

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

message.charAt(0) message.length() is 15 message.charAt(14)

Extracting Substrings

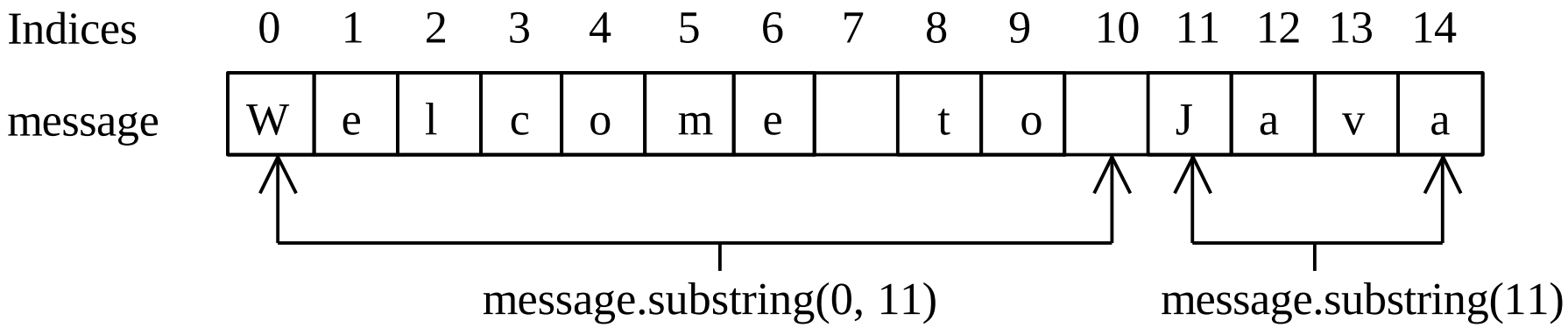
java.lang.String
+subString(beginIndex: int): String +subString(beginIndex: int, endIndex: int): String

Returns this string's substring that begins with the character at the specified beginIndex and extends to the end of the string.

Returns this string's substring that begins at the specified beginIndex and extends to the character at index endIndex – 1. Note that the character at endIndex is not part of the substring.

Extracting Substrings

```
String s1 = "Welcome to Java";
String s2 = s1.substring(0, 11) + "HTML";
```



s2 will be "Welcome to HTML"

Finding a Character or a Substring in a String

java.lang.String	
+indexOf(ch: char): int	Returns the index of the first occurrence of ch in the string. Returns -1 if not matched.
+indexOf(ch: char, fromIndex: int): int	Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matched.
+indexOf(s: String): int	Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
+indexOf(s: String, fromIndex: int): int	Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
+lastIndexOf(ch: int): int	Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
+lastIndexOf(ch: int, fromIndex: int): int	Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
+lastIndexOf(s: String): int	Returns the index of the last occurrence of string s. Returns -1 if not matched.
+lastIndexOf(s: String, fromIndex: int): int	Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.

Finding a Character or a Substring in a String

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

`"Welcome to Java".indexOf('W')` returns 0.

`"Welcome to Java".indexOf('x')` returns -1.

`"Welcome to Java".indexOf('o', 5)` returns 9.

`"Welcome to Java".indexOf("come")` returns 3.

`"Welcome to Java".indexOf("Java", 5)` returns 11.

`"Welcome to Java".indexOf("java", 5)` returns -1.

`"Welcome to Java".lastIndexOf('a')` returns 14.

Converting, Replacing, and Splitting Strings

java.lang.String	
+toLowerCase(): String	Returns a new string with all characters converted to lowercase.
+toUpperCase(): String	Returns a new string with all characters converted to uppercase.
+trim(): String	Returns a new string with blank characters trimmed on both sides.
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Examples

"Welcome".toLowerCase() returns a new string, "welcome".

"Welcome".toUpperCase() returns a new string, "WELCOME".

" Welcome to Java ".trim() returns a new string, "Welcome to Java".

"Welcome".replace('e', 'A') returns a new string, "WAlcomA".

"Welcome".replaceFirst("e", "AB") returns a new string, "WABlcome".

"Welcome".replaceAll("e", "AB") returns a new string, "WABlcomAB".

"Welcome".replaceAll("el", "AB") returns a new string, "WABcome".

Splitting a String

```
String[] tokens = "Java#HTML#Perl".split("#");
for(int i = 0; i < tokens.length; i++)
  System.out.println(tokens[i]);
```

displays

Java

HTML

Perl

Matching, Replacing and Splitting by Patterns

- The replaceAll, replaceFirst, split and matches methods can be used with a *regular expression (a sequence of characters that specifies a search pattern)*
  - Example: the following statement returns a new string that replaces $, +, or # in "a+b$#c" by the string NNN.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");
System.out.println(s);
```

Displays: aNNNbNNNNNNNc

- The regular expression [$+#] specifies a pattern that matches $, +, or #

Matching, Replacing and Splitting by Patterns

java.lang.String	
+matches(regex: String): boolean	Returns true if this string matches the pattern.
+replaceAll(regex: String, replacement: String): String	Returns a new string that replaces all matching substrings with the replacement.
+replaceFirst(regex: String, replacement: String): String	Returns a new string that replaces the first matching substring with the replacement.
+split(regex: String): String[]	Returns an array of strings consisting of the substrings split by the matches.

Matching, Replacing and Splitting by Patterns

- The **`matches`** method tells whether or not a string matches a given regular expression.

```
"Java is fun".matches("Java.*")
true
```

Social security numbers is **xxx-xx-xxxx, where x is a digit:**

[\\d]{3}-[\\d]{2}-[\\d]{4}

An even number ends with digits **0, 2, 4, 6, or 8:**

[\\d]*[02468]

Telephone numbers **(xxx) xxx-xxxx, where x is a digit and the first digit cannot be zero:**

\\([1-9][\\d]{2}\\) [\\d]{3}-[\\d]{4}

Regular Expressions

Regular Expression	Matches	Example
x	a specified character x	Java matches Java
.	any single character	Java matches J..a
(ab|cd)	ab or cd	ten matches t(en|im)
[abc]	a, b, or c	Java matches Ja[uvwx]a
[^abc]	any character except a, b, or c	Java matches Ja[^ars]a
[a-z]	a through z	Java matches [A-M]av[a-d]
[^a-z]	any character except a through z	Java matches Jav[^b-d]
[a-e[m-p]]	a through e or m through p	Java matches [A-G[I-M]]av[a-d]
[a-e&&[c-p]]	intersection of a-e with c-p	Java matches [A-P&&[I-M]]av[a-d]
\d	a digit, same as [1-9]	Java2 matches "Java[\\d]"
\D	a non-digit	$Java matches "[\\D][\\D]ava"
\w	a word character	Java matches "[\\w]ava"
\W	a non-word character	$Java matches "[\\W][\\w]ava"
\s	a whitespace character	"Java 2" matches "Java\\s2"
\S	a non-whitespace char	Java matches "[\\S]ava"
*p**	zero or more occurrences of pattern *p*	Java matches "[\\w]*"
p+	one or more occurrences of pattern *p*	Java matches "[\\w]+"
p?	zero or one occurrence of pattern *p*	Java matches "[\\w]?Java" Java matches "[\\w]?ava"
p{n}	exactly n occurrences of pattern *p*	Java matches "[\\w]{4}"
p{n,}	at least n occurrences of pattern *p*	Java matches "[\\w]{3,}"
p{n,m}	between n and m occurrences (inclusive)	Java matches "[\\w]{1,9}"

Matching, Replacing and Splitting by Patterns

- The following statement splits the string into an array of strings delimited by some punctuation marks:

```
String[] tokens = "Java,C;C#.C++".split("[,;.]");
for (int i = 0; i < tokens.length; i++)
  System.out.println(tokens[i]);
```

Displays:

```
Java
C
C#
C++
```

Examples

```
String s = "Java Java Java".replaceAll("v\\w", "wi") ;
      // “Jawi Jawi Jawi”

String s2 = "Java Java Java".replaceFirst("v\\w", "wi") ;
      // “Jawi Java Java”

String[] s3 = "Java1HTML2Perl".split("\\d");
      // [“Java”, “HTML”, “Perl”]
```

Command-Line Parameters

```
class TestMain {
  public static void main(String[] args){
      ...
  }
}
```

- We can run the program with:

```
java TestMain arg0 arg1 arg2 ... argn
```

or the EclipseIDE Run Configuration arguments

- In the main method, get the arguments `args[0], args[1], ..., args[n]` corresponds to `arg0, arg1, ..., argn` in the command line or the EclipseIDE Run Configuration arguments

Processing Command-Line Parameters

```
public class Calculator {
  public static void main(String[] args) {
      if (args.length != 3) {
            System.out.println("Usage: java Calculator
                  operand1 operator operand2");
            System.exit(0);
      }
      int result = 0;
      switch (args[1].charAt(0)) {
      case '+':     result = Integer.parseInt(args[0])+
                          Integer.parseInt(args[2]);
                    break;
      ...
  }
}
```

```
javac Calculator.java
java Calculator 1 + 2
3
```

StringBuilder and StringBuffer

- The **StringBuilder/StringBuffer** classes are alternatives to the **String** class:
  - **StringBuilder/StringBuffer** can be used wherever a string is used
    - **StringBuffer** is synchronized i.e. thread safe. It means two threads can't call the methods of **StringBuffer** simultaneously. **StringBuilder** is non-synchronized i.e. not thread safe. It means two threads can call the methods of StringBuilder simultaneously.
  - **StringBuilder/StringBuffer** is more flexible than **String**
    - You can **add**, **insert**, or **append** new contents into a string buffer, whereas the value of a **String** object is fixed once the string is created

StringBuilder Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

Modifying StringBuilder(s)

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: *aPrimitiveType*): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: *aPrimitiveType*): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Modifying StringBuilder(s)

```
                                                StringBuilder:
StringBuilder stringBuilder =
                        new StringBuilder();
stringBuilder.append("Java");                   Java
stringBuilder.insert(2,"HTML and ");            JaHTML and va
stringBuilder.delete(3, 4);                     JaHML and va
stringBuilder.deleteCharAt(5);                  JaHMLand  va
stringBuilder.reverse();                        av dnaLMHaJ
stringBuilder.replace(4, 8, "HTML");            av dHTMLHaJ
stringBuilder.setCharAt(0, 'w');                wv dHTMLHaJ
```

The `toString`, `capacity`, `length`, `setLength`, and `charAt` Methods

java.lang.StringBuilder	
+toString(): String	Returns a string object from the string builder.
+capacity(): int	Returns the capacity of this string builder.
+charAt(index: int): char	Returns the character at the specified index.
+length(): int	Returns the number of characters in this builder.
+setLength(newLength: int): void	Sets a new length in this builder.
+substring(startIndex: int): String	Returns a substring starting at startIndex.
+substring(startIndex: int, endIndex: int): String	Returns a substring from startIndex to endIndex-1.
+trimToSize(): void	Reduces the storage size used for the string builder.

The `Character` Class

java.lang.Character	
+Character(value: char)	Constructs a character object with char value
+charValue(): char	Returns the char value from this object
+compareTo(anotherCharacter: Character): int	Compares this character with another
+equals(anotherCharacter: Character): boolean	Returns true if this character equals to another
+isDigit(ch: char): boolean	Returns true if the specified character is a digit
+isLetter(ch: char): boolean	Returns true if the specified character is a letter
+isLetterOrDigit(ch: char): boolean	Returns true if the character is a letter or a digit
+isLowerCase(ch: char): boolean	Returns true if the character is a lowercase letter
+isUpperCase(ch: char): boolean	Returns true if the character is an uppercase letter
+toLowerCase(ch: char): char	Returns the lowercase of the specified character
+toUpperCase(ch: char): char	Returns the uppercase of the specified character

equals and compareTo

```
Character charObject = new Character('b');

charObject.equals(new Character('b'))
     returns true
charObject.equals(new Character('d'))
     returns false

charObject.compareTo(new Character('a'))
     returns 1
charObject.compareTo(new Character('b'))
     returns 0
charObject.compareTo(new Character('c'))
     returns -1
charObject.compareTo(new Character('d')
     returns -2
```

Designing Classes

- Coherence: A class should describe a single entity
  - Separating responsibilities: A single entity with too many responsibilities can be broken into several classes to separate responsibilities
- Reuse: Classes are designed for reuse!

Designing Classes

- Follow standard Java programming style and naming conventions:
  - Choose informative names for classes, data fields, and methods
- Place the data declaration before the constructor, and place constructors before methods.
- Provide a public no-arg constructor and override the **`equals`** method and the **`toString`** method (returns a **`String`**) whenever possible